

Core Language Working Group "ready" Issues for the July, 2017 (Toronto) meeting

Section references in this document reflect the section numbering of document [WG21 N4659](#).

943. Is $\tau()$ a temporary?

Section: 8.2.3 [expr.type.conv] **Status:** ready **Submitter:** Miller **Date:** 14 July, 2009

According to 8.2.3 [expr.type.conv] paragraphs 1 and 3 (stated directly or by reference to another section of the Standard), all the following expressions create temporaries:

```
 $\tau(1)$   
 $\tau(1, 2)$   
 $\tau\{1\}$   
 $\tau\{\}$ 
```

However, paragraph 2 says,

The expression $\tau()$, where τ is a *simple-type-specifier* or *typename-specifier* for a non-array complete effective object type or the (possibly cv-qualified) `void` type, creates an rvalue of the specified type, which is value-initialized (11.6 [dcl.init]; no initialization is done for the `void()` case).

This does *not* say that the result is a temporary, which means that the lifetime of the result is not specified by 15.2 [class.temporary]. Presumably this is just an oversight.

Notes from the October, 2009 meeting:

The specification in 8.2.3 [expr.type.conv] is in error, not because it fails to state that $\tau()$ is a temporary but because it requires a temporary for the other cases with fewer than two operands. The case where τ is a class type is covered by 15.2 [class.temporary] paragraph 1 (“a conversion that creates an rvalue”), and a temporary should *not* be created when τ is not a class type.

Proposed resolution (March, 2017):

This issue is resolved by the resolution of issue 1299.

1076. Value categories and lvalue temporaries

Section: 6.10 [basic.lval] **Status:** ready **Submitter:** Daniel Krügler **Date:** 2010-06-10

The taxonomy of value categories in 6.10 [basic.lval] classifies temporaries as prvalues. However, some temporaries are explicitly referred to as lvalues (cf 18.1 [except.throw] paragraph 3).

Proposed resolution (March, 2017):

This issue is resolved by the resolution of issue 1299.

1299. “Temporary objects” vs “temporary expressions”

Section: 15.2 [class.temporary] **Status:** ready **Submitter:** Johannes Schaub **Date:** 2011-04-16

The Standard is insufficiently precise in dealing with temporaries. It is not always clear when the term “temporary” is referring to an expression whose result is a prvalue and when it is referring to a temporary object.

(See also issue 1568.)

Proposed resolution (February, 2014) [SUPERSEDED]:

The resolution is contained in document N3918.

This resolution also resolves issues 1651 and 1893.

Additional note, November, 2014:

Concerns have been raised that the meaning of “corresponding temporary object” is not clear enough in the proposed wording. In addition, N3918 says that it resolves issue 1300, but that issue is 1) marked as a duplicate of issue 914 and 2) the subject of continuing deliberations in EWG. This issue is being returned to "review" status to allow CWG to address these concerns.

Proposed resolution (March, 2017):

1. Change 8 [expr] paragraph 12 as follows:

...is applied. [Note: If the expression is an lvalue of class type, it must have a volatile copy constructor to initialize the temporary **object** that is the result object of the lvalue-to-rvalue conversion. —end note] The glvalue expression...

2. Change 15.2 [class.temporary] paragraph 6 as follows:

The third context is when a reference is bound to a temporary **object**.¹¹⁶ The temporary **object** to which the reference is bound or the temporary **object** that is the complete object of a subobject to which the reference is bound persists for the lifetime of the reference **if the glvalue referring to the temporary object was obtained through one of the following:**

- **a temporary materialization conversion (7.4 [conv.rval]),**
- **(*expression*), where *expression* is one of these expressions,**
- **subscripting (8.2.1 [expr.sub]) of an array operand, where that operand is one of these expressions,**
- **a class member access (8.2.5 [expr.ref]) using the . operator where the left operand is one of these expressions and the right operand designates a non-static data member of non-reference type,**
- **a pointer-to-member operation (8.5 [expr.mptr.oper]) using the . * operator where the left operand is one of these expressions and the right operand is a pointer to data member of non-reference type,**
- **a `const_cast` (8.2.11 [expr.const.cast], `static_cast` (8.2.9 [expr.static.cast]), `dynamic_cast` (8.2.7 [expr.dynamic.cast]), or `reinterpret_cast` (8.2.10 [expr.reinterpret.cast]) converting a**

glvalue operand that is one of these expressions to a glvalue referring to that operand,

- a conditional expression (8.16 [expr.cond]) that is a glvalue where the second or third operand is one of these expressions, or
- a comma expression (8.19 [expr.comma]) that is a glvalue where the right operand is one of these expressions.

[Example:

```
template<typename T> using id = T;
```

```
int&& a = id<int[3]>{1, 2, 3}[i];           // temporary array has same lifetime as a
```

```
const int& b = static_cast<const int&>(0); // temporary int has same lifetime as b
```

```
int&& c = cond ? id<int[3]>{1, 2, 3}[i] : static_cast<int&&>(0);
```

```
// exactly one of the two temporaries is lifetime-extended
```

—end example] [Note: If a temporary object has a reference member initialized by another temporary object, lifetime extension applies recursively to such a member's initializer. [Example:

```
struct S {  
    const int& m;  
};
```

```
const S& s = S{1}; // both s and int temporaries have lifetime of s
```

—end example] —end note]

except The exceptions to this lifetime rule are:

- A temporary object bound to a reference parameter...

3. Change 16.3.1.4 [over.match.copy] bullet 1.2 as follows:

- When the type of the initializer expression is a class type “cv S”, the non-explicit conversion functions of S and its base classes are considered. When initializing a temporary **object** (12.2 [class.mem]) to be bound to the first parameter of a constructor...

4. Change 20.5.4.9 [res.on.arguments] bullet 1.3 as follows:

- ...[Note: If a program casts an lvalue to an xvalue while passing that lvalue to a library function (e.g. by calling the function with the argument `std::move(x)`), the program is effectively asking that function to treat that lvalue as a temporary **object**. The implementation is free...

5. Change 23.11.2.3.3 [util.smartptr.weak.assign] paragraph 2 as follows:

Remarks: The implementation may meet the effects (and the implied guarantees) via different means, without creating a temporary **object**.

6. Change the footnote in 29.7.2.1 [template.valarray.overview] paragraph 1 as follows:

...generalized subscript operators. [Footnote: The intent is to specify an array template that has the minimum functionality necessary to address aliasing ambiguities and the proliferation of **temporaries** **temporary objects**. Thus... —end footnote]

7. Change the last bullet of C.2.16 [diff.cpp03.input.output] as follows:

- initializing a `const bool &` which would bind to a temporary **object**.

This resolution also resolves issues 943 and 1076.

2253. Unnamed bit-fields and zero-initialization

Section: 12.2.4 [class.bit] **Status:** ready **Submitter:** Aaron Ballman **Date:** 2016-03-23

The current wording of the Standard does not clearly state that zero-initialization applies to unnamed bit-fields.

Notes from the December, 2016 teleconference:

The consensus was that unnamed bit-fields do constitute padding; more generally, padding should be normatively defined, along the lines suggested in 12.2.4 [class.bit] paragraphs 1-2.

Proposed resolution (March, 2017):

1. Change 6.9 [basic.types] paragraph 4 as follows:

The *object representation* of an object of type τ is the sequence of N unsigned char objects taken up by the object of type τ , where N equals `sizeof( $\tau$ )`. The *value representation* of an object is the set of bits that hold the value of type τ . **Bits in the object representation that are not part of the value representation are padding bits.** For trivially copyable types, the value representation is a set of bits in the object representation that determines a *value*, which is one discrete element of an implementation-defined set of values.⁴⁴

2. Change 11.6 [dcl.init] paragraph 6 as follows:

To *zero-initialize* an object or reference of type τ means:

- if τ is a scalar type (6.9 [basic.types]), the object is initialized to the value obtained by converting the integer literal 0 (zero) to τ ;¹⁰³
- if τ is a (possibly cv-qualified) non-union class type, **its padding bits are initialized to zero bits and** each non-static data member and each base-class subobject is zero-initialized ~~and padding is initialized to zero bits~~;
- if τ is a (possibly cv-qualified) union type, **its padding bits are initialized to zero bits and** the object's first non-static named data member is zero-initialized ~~and padding is initialized to zero bits~~;
- ...

3. Change 12.2.4 [class.bit] paragraph 1 as follows:

...The *constant-expression* shall be an integral constant expression with a value greater than or equal to zero. The value of the integral constant expression may be larger than the number of bits in the object representation (6.9 [basic.types]) of the bit-field's type; in such cases the extra bits are ~~used as padding bits~~ **(6.9 [basic.types])** ~~and do not participate in the value representation (6.9 [basic.types]) of the bit field.~~ Allocation of bit-fields...

4. Change 6.9.1 [basic.fundamental] paragraph 1 as follows:

...For narrow character types, all bits of the object representation participate in the value representation. [Note: A bit-field of narrow character type whose length is larger than the number of bits in the object representation of that type has padding bits; see ~~12.2.4 [class.bit]~~ **6.9 [basic.types]**. —end note] For unsigned narrow character types...